

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования

**“ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО”**

Факультет прикладной информатики (фПИИ)

Направление подготовки (специальность) – 45.03.04 (Языковые модели и
Искусственный Интеллект)

Алгоритмы и структуры данных

Лабораторная работа № 4

Вариант 21

Выполнил студент: Попов Глеб Ильич Группа № К3260

Преподаватель: Харлуниин Александр Александрович

г. Санкт-Петербург 2026 г

Задача №1. Наивный поиск подстроки в строке.....	4
Задача №2. Карта.....	7
Задача №3. Паттерн в тексте.....	12
Задача №4. Равенство подстрок.....	17
Задача №5. Префикс-функция.....	23
Задача №6. Z-функция.....	27
Задача №7. Наибольшая общая подстрока.....	31
Задача №8. Шаблоны с несовпадениями.....	38
Задача №9. Декомпозиция строки.....	47
Дополнительная задача №1. Подстроки из одинаковых букв.....	55
Дополнительная задача №2. Количество различных подстрок.....	60
Общий вывод по работе.....	65

Задача №1. Наивный поиск подстроки в строке

Текст задачи:

Даны строки p и t . Требуется найти все вхождения строки p в строку t в качестве подстроки.

В первой строке входного файла находится строка p , во второй строке - строка t . Строки состоят из букв латинского алфавита.

Нужно вывести количество вхождений строки p в строку t , а затем во второй строке вывести позиции, с которых начинаются эти вхождения. Нумерация символов начинается с единицы.

Ограничения:

$$1 \leq |p|, |t| \leq 10^4$$

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        lines = f.read().splitlines()

    p = lines[0]

    t = lines[1]

    result = []

    len_p = len(p)

    len_t = len(t)

    for i in range(len_t - len_p + 1):

        ok = True

        for j in range(len_p):

            if t[i + j] != p[j]:

                ok = False

                break

        if ok:

            result.append(i + 1)
```

```

with open("output.txt", "w", encoding="utf-8") as f:

    f.write(str(len(result)) + "\n")

    if result:

        f.write(" ".join(map(str, result)) + "\n")

    else:

        f.write("\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче необходимо найти все позиции, с которых строка *p* входит в строку *t*.

Для решения используется наивный алгоритм поиска подстроки.

Программа перебирает все возможные позиции начала подстроки в строке

t. Для каждой позиции проверяется, совпадают ли все символы строки p с соответствующими символами строки t.

Если все символы совпали, значит найдено очередное вхождение. В ответ добавляется позиция начала этого вхождения. Так как в условии задачи нумерация начинается с единицы, к индексу i прибавляется 1.

После проверки всех возможных позиций программа выводит количество найденных вхождений и сами позиции в возрастающем порядке.

Сложность алгоритма составляет $O(|p| * |t|)$, так как для каждой позиции в тексте может потребоваться сравнить все символы образца. При ограничениях до 10^4 такой подход подходит для данной задачи.

Таблица тестирования:

время	память	input	output
0.001688 sec	10.35 KB	aba abaCaba	2 1 5
0.001025 sec	10.34 KB	aa aaaaa	4 1 2 3 4
0.001055 sec	10.31 KB	abc defgh	0
0.001056 sec	10.30 KB	a banana	3 2 4 6

Вывод по задаче

В ходе решения задачи был реализован наивный алгоритм поиска подстроки в строке. Программа последовательно проверяет каждую возможную позицию начала образца в тексте и сохраняет все найденные совпадения. Решение корректно обрабатывает случаи с несколькими вхождениями, пересекающимися вхождениями и полным отсутствием

совпадений. Алгоритм прост в реализации и укладывается в ограничения задачи.

Задача №2. Карта

Текст задачи:

В послании на карте спрятано число x . Чтобы его найти, нужно удалить все пробелы между словами, а затем посчитать количество способов вычеркнуть все буквы кроме трех так, чтобы оставшееся слово из трех букв одинаково читалось слева направо и справа налево.

То есть после удаления пробелов нужно найти количество подпоследовательностей длины 3 вида:

a b a

где первая и третья буквы совпадают.

Во входном файле дана одна строка - послание, состоящее из строчных английских букв и пробелов. Нужно вывести одно число x - количество способов оставить три буквы так, чтобы получился палиндром длины 3. Длина послания не превышает $3 \cdot 10^5$, а пробелы перед подсчетом нужно удалить.

Код:

```
import time

import tracemalloc

def solve():
```

```
with open("input.txt", "r", encoding="utf-8") as f:

    s = f.read().strip()

s = s.replace(" ", "")

right = [0] * 26

left = [0] * 26

for ch in s:

    right[ord(ch) - ord("a")] += 1

ans = 0

for ch in s:

    c = ord(ch) - ord("a")

    right[c] -= 1

    for i in range(26):

        ans += left[i] * right[i]

    left[c] += 1

with open("output.txt", "w", encoding="utf-8") as f:

    f.write(str(ans) + "\n")

tracemalloc.start()
```



```
start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()
cur, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо посчитать количество способов выбрать из строки три символа так, чтобы первый и третий символы совпадали. Перед обработкой из строки удаляются все пробелы, так как по условию они не должны учитываться.

Идея решения заключается в том, чтобы перебирать каждую букву строки как средний символ будущего палиндрома длины 3. Для каждой позиции нужно знать, сколько одинаковых букв находится слева и справа от нее.

Для этого используются два массива длины 26:

left - количество каждой буквы слева от текущей позиции;

right - количество каждой буквы справа от текущей позиции.

Сначала массив right заполняется количеством всех букв в строке. Затем программа идет по строке слева направо. Перед обработкой текущего символа он удаляется из массива right, потому что текущая буква является серединой и уже не находится справа.

После этого для каждой буквы английского алфавита считается количество вариантов выбрать такую букву слева и такую же букву справа. Если некоторой буквы слева $left[i]$, а справа $right[i]$, то она дает:

$left[i] * right[i]$ палиндромов с текущей буквой в середине.

После подсчета текущий символ добавляется в массив $left$, так как при переходе к следующей позиции он уже будет находиться слева.

Алгоритм работает за $O(26 * n)$, что фактически является линейной сложностью. Память используется постоянная, так как хранятся только два массива по 26 элементов.

Таблица тестирования:

время	память	input	output
0.001825 sec	10.65 KB	treasure	8
0.002541 sec	10.67 KB	you will never find the treasure	146
0.001491 sec	10.64 KB	aaa	1
0.001182 sec	10.65 KB	aaaa	4

Пояснение к тестам

В первом тесте строка `treasure`. В ней можно выбрать 8 троек символов, которые после вычеркивания остальных букв образуют палиндром длины 3.

Во втором тесте пробелы сначала удаляются, после чего программа считает все подходящие подпоследовательности.

В третьем тесте строка aaa. Можно оставить только одну тройку символов, поэтому ответ равен 1.

В четвертом тесте строка aaaa. Любые три символа из четырех образуют палиндром aaa, поэтому ответ равен $C(4, 3) = 4$.

Вывод по задаче

В ходе решения задачи был реализован алгоритм подсчета палиндромных подпоследовательностей длины 3. Программа удаляет пробелы из исходного послания, после чего для каждой позиции рассматривает текущую букву как середину палиндрома. За счет хранения количества букв слева и справа удается быстро подсчитать количество вариантов для каждой позиции. Решение корректно работает на строках с пробелами, повторяющимися буквами и обычными текстовыми сообщениями, а также укладывается в ограничения по времени и памяти.

Задача №3. Паттерн в тексте

Текст задачи:

Необходимо реализовать алгоритм Рабина-Карпа для поиска заданного шаблона в заданном тексте.

На вход подаются две строки:

P

T

где P - паттерн, а T - текст.

Нужно найти все вхождения строки P в строку T в качестве подстроки.

В первой строке вывода требуется вывести количество вхождений, а во второй строке - позиции, с которых начинаются найденные вхождения. Символы нумеруются с единицы.

Паттерн и текст содержат только латинские буквы. Вхождения могут пересекаться, и их все равно нужно выводить. Также поиск чувствителен к

регистру, поэтому, например, Test и test считаются разными строками.
Ограничения: $1 \leq |P|, |T| \leq 10^6$

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        lines = f.read().splitlines()

    pattern = lines[0]

    text = lines[1]

    n = len(text)

    m = len(pattern)

    if m > n:

        with open("output.txt", "w", encoding="utf-8") as f:

            f.write("0\n\n")

        return

    mod = 1_000_000_007

    x = 263
```

```

power = [1] * (n + 1)

for i in range(1, n + 1):

    power[i] = (power[i - 1] * x) % mod

pref = [0] * (n + 1)

for i in range(n):

    pref[i + 1] = (pref[i] * x + ord(text[i])) % mod

pattern_hash = 0

for ch in pattern:

    pattern_hash = (pattern_hash * x + ord(ch)) % mod

result = []

for i in range(n - m + 1):

    current_hash = (pref[i + m] - pref[i] * power[m]) % mod

    if current_hash == pattern_hash:

        if text[i:i + m] == pattern:

            result.append(i + 1)

with open("output.txt", "w", encoding="utf-8") as f:

    f.write(str(len(result)) + "\n")

    if result:

        f.write(" ".join(map(str, result)) + "\n")

    else:

        f.write("\n")

```

```
tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо найти все вхождения паттерна P в текст T . Так как длины строк могут достигать 10^6 , простой перебор всех позиций с полным сравнением строк может работать слишком медленно. Поэтому используется алгоритм Рабина-Карпа.

Сначала вычисляется хеш паттерна. Затем для текста строится массив префиксных хешей. С его помощью можно быстро получать хеш любой подстроки текста нужной длины.

После этого программа перебирает все возможные позиции начала паттерна в тексте. Для каждой позиции вычисляется хеш подстроки текста длины m , где m — длина паттерна. Если этот хеш совпадает с хешем

паттерна, то выполняется дополнительная проверка через обычное сравнение строк:

```
text[i:i + m] == pattern
```

Эта проверка нужна, потому что разные строки теоретически могут иметь одинаковые хеши. Если строки действительно совпали, позиция добавляется в ответ. Так как индексация в задаче начинается с единицы, в ответ добавляется $i + 1$.

Если длина паттерна больше длины текста, вхождений быть не может, поэтому программа сразу выводит 0.

Алгоритм работает примерно за $O(|T| + |P|)$, если не учитывать редкие дополнительные сравнения при совпадении хешей. Память также используется линейно относительно длины текста.

Таблица тестирования:

время	память	input	output
0.001810 sec	10.94 KB	aba abacaba	2 1 5
0.001779 sec	11.34 KB	Test testTesttesT	1 5
0.001571 sec	11.02 KB	aaaaa baaaaaaa	3 2 3 4
0.001416 sec	11.26 KB	abc ababcabcabc	3 3 6 9

Пояснение к тестам

В первом тесте паттерн aba встречается в тексте abacaba два раза: с позиции 1 и с позиции 5.

Во втором тесте проверяется чувствительность к регистру. Паттерн Test встречается только один раз, начиная с позиции 5.

В третьем тесте проверяется ситуация с пересекающимися вхождениями. Паттерн аaaaaa встречается в строке бааааааа с позиций 2, 3 и 4.

В четвертом тесте паттерн abc встречается в тексте несколько раз, поэтому программа должна вывести все позиции.

Вывод по задаче

В ходе решения задачи был реализован алгоритм Рабина-Карпа для поиска паттерна в тексте. Для быстрого сравнения подстроки используются полиномиальные хеши и массив префиксных хешей. При совпадении хешей дополнительно выполняется обычное сравнение строк, что исключает ошибку из-за возможной коллизии. Программа корректно обрабатывает разные регистры символов, пересекающиеся вхождения и случай, когда паттерн длиннее текста.

Задача №4. Равенство подстрок

Текст задачи:

Дана строка s , состоящая из строчных латинских букв. После этого дано число запросов q . Каждый запрос задается тремя числами:

$a \ b \ l$

Нужно проверить, равны ли две подстроки строки s длины l :

$s[a \dots a + l - 1]$

$s[b \dots b + l - 1]$

Индексы a и b начинаются с нуля.

Для каждого запроса нужно вывести:

Yes

если подстроки равны, и

No

если они различаются.

Ограничения:

$$1 \leq |s| \leq 500000$$

$$1 \leq q \leq 100000$$

$$0 \leq a, b \leq |s| - 1$$

Код:

```
import time

import tracemalloc

def get_hash(pref, power, left, length, mod):

    return (pref[left + length] - pref[left] * power[length]) % mod

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        data = f.read().split()

    s = data[0]

    q = int(data[1])

    n = len(s)
```

```
mod1 = 1_000_000_007

mod2 = 1_000_000_009

x = 263


power1 = [1] * (n + 1)

power2 = [1] * (n + 1)


for i in range(1, n + 1):

    power1[i] = (power1[i - 1] * x) % mod1

    power2[i] = (power2[i - 1] * x) % mod2


pref1 = [0] * (n + 1)

pref2 = [0] * (n + 1)


for i in range(n):

    value = ord(s[i])

    pref1[i + 1] = (pref1[i] * x + value) % mod1

    pref2[i + 1] = (pref2[i] * x + value) % mod2


result = []

pos = 2


for _ in range(q):

    a = int(data[pos])

    b = int(data[pos + 1])

    length = int(data[pos + 2])
```

```

pos += 3

h1_a = get_hash(pref1, power1, a, length, mod1)
h1_b = get_hash(pref1, power1, b, length, mod1)

h2_a = get_hash(pref2, power2, a, length, mod2)
h2_b = get_hash(pref2, power2, b, length, mod2)

if h1_a == h1_b and h2_a == h2_b:
    result.append("Yes")
else:
    result.append("No")

with open("output.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(result) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

```

```
f.write(f"time = {finish_time - start_time:.6f} sec\n")

f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо быстро отвечать на запросы о равенстве двух подстрок одной строки. Если каждый раз сравнивать подстроки напрямую, то при больших ограничениях программа может работать слишком долго. Поэтому используется полиномиальное хеширование.

Сначала для строки s вычисляются массивы степеней основания x и массивы префиксных хешей. Для уменьшения вероятности коллизий используются два разных модуля:

$mod1 = 1_000_000_007$

$mod2 = 1_000_000_009$

Префиксный хеш позволяет получить хеш любой подстроки за $O(1)$. Для подстроки, которая начинается с позиции $left$ и имеет длину $length$, хеш вычисляется по формуле:

$$(pref[left + length] - pref[left] * power[length]) \% mod$$

После предварительной обработки программа обрабатывает каждый запрос. Для двух подстрок вычисляются хеши по первому модулю и по второму модулю. Если оба хеша совпадают, программа считает подстроки равными и выводит Yes. Если хотя бы по одному модулю хеши отличаются, выводится No.

В отличие от задачи Рабина-Карпа, здесь подстроки дополнительно напрямую не сравниваются, потому что по условию задачи предполагается использование двойного хеширования. Вероятность ошибки при двух больших модулях крайне мала.

Сложность предварительной обработки составляет $O(n)$, где n - длина строки. Каждый запрос обрабатывается за $O(1)$. Общая сложность решения - $O(n + q)$, что подходит под ограничения задачи.

Таблица тестирования:

время	память	input	output
0.001942 sec	11.62 KB	trololo 4 0 0 7 2 4 3 3 5 1 1 3 2	Yes Yes Yes No
0.001210 sec	11.12 KB	aaaaa 4 0 1 4 0 2 3 1 3 2 2 4 1	Yes Yes Yes Yes
0.001290 sec	11.46 KB	abcdef 4 0 3 2 1 1 3 2 4 1 0 0 6	No Yes No Yes
0.001951 sec	11.65 KB	abacaba 5 0 4 3 0 2 2 1 5 1 2 6 1 3 3 4	Yes No Yes Yes Yes

Пояснение к тестам

В первом тесте используется пример из условия. Подстроки в первых трех запросах равны, а в последнем запросе подстроки различаются.

Во втором тесте вся строка состоит из одинаковых символов, поэтому любые подстроки одинаковой длины равны.

В третьем тесте проверяется строка с разными символами. В ней равны только подстроки, которые начинаются с одной и той же позиции или полностью совпадают.

В четвертом тесте проверяется строка abacaba. Например, запрос 0 4 3 сравнивает подстроки aba и aba, поэтому ответ Yes.

Вывод по задаче

В ходе решения задачи был реализован алгоритм быстрого сравнения подстрок с помощью полиномиального хеширования. После предварительного вычисления степеней и префиксных хешей каждая пара подстрок сравнивается за константное время. Использование двух модулей уменьшает вероятность коллизий. Решение эффективно обрабатывает большое количество запросов и укладывается в ограничения по времени и памяти.

Задача №5. Префикс-функция

Текст задачи:

Дана строка s , состоящая из букв латинского алфавита. Нужно построить префикс-функцию для всех непустых префиксов этой строки.

Префикс-функция для позиции i показывает длину наибольшего собственного префикса строки $s[0...i]$, который одновременно является ее суффиксом.

Собственный префикс - это префикс, который не равен всей строке целиком.

Нужно вывести значения префикс-функции для всех префиксов строки s длиной от 1 до $|s|$.

Ограничения:

$$1 \leq |s| \leq 10^6$$

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        s = f.read().strip()

    n = len(s)

    pref = [0] * n

    for i in range(1, n):

        j = pref[i - 1]

        while j > 0 and s[i] != s[j]:

            j = pref[j - 1]

        if s[i] == s[j]:

            j += 1

        pref[i] = j

    with open("output.txt", "w", encoding="utf-8") as f:
```

```

        f.write(" ".join(map(str, pref)) + "\n")

    tracemalloc.start()

    start_time = time.perf_counter()

    solve()

    finish_time = time.perf_counter()

    cur, peak = tracemalloc.get_traced_memory()

    tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче необходимо для каждой позиции строки найти значение префикс-функции. Значение $\text{pref}[i]$ равно длине наибольшего собственного префикса подстроки $s[0...i]$, который также является суффиксом этой подстроки.

Решение строится слева направо. Для первой позиции значение всегда равно 0, потому что у строки длины 1 нет собственного непустого префикса.

Для каждой следующей позиции i сначала берется значение $j = \text{pref}[i - 1]$. Это длина наибольшего подходящего префикса для предыдущего префикса строки. Затем программа пытается продолжить это совпадение текущим символом $s[i]$.

Если символы $s[i]$ и $s[j]$ не совпадают, то текущий кандидат не подходит. Тогда программа переходит к более короткому возможному префиксу:

$j = \text{pref}[j - 1]$

Такой переход повторяется, пока либо символы не совпадут, либо j не станет равным нулю.

Если после этого символы совпали, значение j увеличивается на 1. Полученное значение записывается в $\text{pref}[i]$.

Благодаря переходам по уже вычисленным значениям префикс-функции каждый символ обрабатывается эффективно. Общая сложность алгоритма составляет $O(n)$, где n - длина строки. Память также используется линейно, так как хранится массив значений префикс-функции.

Таблица тестирования:

время	память	input	output
0.001096 sec	10.29 KB	aaaAAA	0 1 2 0 0 0
0.001337 sec	10.30 KB	abacaba	0 0 1 0 1 2 3
0.000926 sec	10.28 KB	aaaaa	0 1 2 3 4
0.001721 sec	10.30 KB	abcabcd	0 0 0 1 2 3 0

Пояснение к тестам

В первом тесте строка aaaAAA. Для первых трех символов значения растут: 0 1 2, потому что префиксы состоят из повторяющихся букв а. После появления заглавной А совпадение сбрасывается, так как регистр символов учитывается.

Во втором тесте строка abacaba. Для последнего символа значение равно 3, потому что у всей строки есть общий префикс и суффикс aba.

В третьем тесте строка состоит только из одинаковых символов, поэтому значение префикс-функции на каждой следующей позиции увеличивается на 1.

В четвертом тесте строка `abcsabcd`. Для префикса `abcsabc` значение равно 3, потому что префикс `abc` совпадает с суффиксом `abc`. Последний символ `d` ломает это совпадение.

Вывод по задаче

В ходе решения задачи был реализован линейный алгоритм построения префикс-функции. Программа последовательно вычисляет значения для всех позиций строки и при несовпадении символов переходит к более коротким уже найденным границам. Такой подход позволяет избежать повторного сравнения одних и тех же фрагментов строки. Решение корректно обрабатывает строки с повторяющимися символами, строки без совпадающих префиксов и суффиксов, а также учитывает различие регистра символов.

Задача №6. Z-функция

Текст задачи:

Дана строка s , состоящая из букв латинского алфавита. Нужно построить Z-функцию для этой строки.

Z-функция для позиции i показывает длину наибольшей подстроки, которая начинается с позиции i и совпадает с префиксом всей строки.

Для нулевой позиции значение обычно равно длине всей строки, но в этой задаче его выводить не нужно. Нужно вывести значения Z-функции для индексов:

1, 2, ..., $|s| - 1$

Ограничения:

$$2 \leq |s| \leq 10^6$$

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        s = f.read().strip()

    n = len(s)

    z = [0] * n

    left = 0

    right = 0

    for i in range(1, n):

        if i <= right:

            z[i] = min(right - i + 1, z[i - left])

            while i + z[i] < n and s[z[i]] == s[i + z[i]]:

                z[i] += 1

            if i + z[i] - 1 > right:

                left = i

                right = i + z[i] - 1
```

```

with open("output.txt", "w", encoding="utf-8") as f:

    f.write(" ".join(map(str, z[1:])) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче необходимо для каждой позиции строки определить, сколько символов начиная с этой позиции совпадает с началом всей строки.

Например, если строка начинается с aba, и с некоторой позиции тоже начинается фрагмент aba, то значение Z-функции для этой позиции будет не меньше 3.

Наивно можно было бы для каждой позиции сравнивать символы с началом строки, но при длине до 10^6 такой подход может быть слишком медленным. Поэтому используется линейный алгоритм построения Z-функции.

В алгоритме поддерживается отрезок [left, right]. Это самый правый найденный отрезок строки, который совпадает с префиксом строки. Если текущая позиция i находится внутри этого отрезка, то часть ответа для нее уже можно взять из ранее посчитанных значений:

$$z[i] = \min(\text{right} - i + 1, z[i - \text{left}])$$

После этого программа пытается расширить совпадение обычным сравнением символов:

```
while i + z[i] < n and s[z[i]] == s[i + z[i]]:
```

```
    z[i] += 1
```

Если новое совпадение вышло правее текущего отрезка [left, right], границы обновляются.

В конце программа выводит значения $z[1:]$, потому что значение для позиции 0 по условию задачи не выводится.

Алгоритм работает за $O(n)$, так как правая граница отрезка только сдвигается вправо и не возвращается назад. Память используется линейно для хранения массива Z-функции.

.

Таблица тестирования:

время	память	input	output
0.001309 sec	10.30 KB	aaabaab	2 1 0 2 1 0
0.001909 sec	10.30 KB	abacaba	0 1 0 3 0 1
0.001545 sec	10.29 KB	zzzzzz	5 4 3 2 1
0.002214 sec	10.34 KB	aabcaabxaaaz	1 0 0 3 1 0 0 2 2 1 0

Пояснение к тестам

В первом тесте строка `aaabaab`. На позиции 1 с началом строки совпадают два символа `aa`, на позиции 2 совпадает один символ `a`. Далее встречается фрагмент `aa`, который снова частично совпадает с префиксом строки.

Во втором тесте строка `abacaba`. На позиции 4 начинается фрагмент `aba`, который совпадает с префиксом строки длины 3.

В третьем тесте строка `zzzzzz` состоит только из одинаковых символов. Поэтому с каждой позиции совпадает максимально возможное количество символов с началом строки: 5 4 3 2 1.

В четвертом тесте строка `aabsaabxaaaz`. В ней есть несколько частичных совпадений с префиксом `aab`, поэтому значения Z -функции ненулевые в нескольких позициях.

Вывод по задаче

В ходе решения задачи был реализован линейный алгоритм построения Z -функции. Программа использует уже найденный правый отрезок совпадения с префиксом строки, поэтому не сравнивает одни и те же символы много раз. Решение корректно обрабатывает строки с повторяющимися символами, строки с частичным совпадением префикса внутри строки и строки без повторений. Алгоритм укладывается в ограничения по времени и памяти.

Задача №7. Наибольшая общая подстрока

Текст задачи:

Даны две строки s и t . Нужно найти строку максимальной длины, которая является подстрокой и строки s , и строки t .

Во входном файле может быть несколько строк. Каждая строка входных данных содержит две строки:

$s\ t$

Для каждой пары строк нужно вывести три числа:

$i\ j\ l$

где:

i - начальная позиция найденной подстроки в строке s ,

j - начальная позиция найденной подстроки в строке t ,

l - длина наибольшей общей подстроки.

Индексация начинается с нуля. Если подходящих подстрок максимальной длины несколько, можно вывести любую.

Код:

```
import time

import tracemalloc

MOD1 = 1000000007

MOD2 = 1000000009

X = 263

def build_hashes(s):

    n = len(s)
```

```

power1 = [1] * (n + 1)

power2 = [1] * (n + 1)

pref1 = [0] * (n + 1)
pref2 = [0] * (n + 1)

for i in range(1, n + 1):
    power1[i] = (power1[i - 1] * X) % MOD1
    power2[i] = (power2[i - 1] * X) % MOD2

for i in range(n):
    value = ord(s[i])
    pref1[i + 1] = (pref1[i] * X + value) % MOD1
    pref2[i + 1] = (pref2[i] * X + value) % MOD2

return pref1, pref2, power1, power2

def get_hash(pref, power, left, length, mod):
    return (pref[left + length] - pref[left] * power[length]) % mod

def check(length, s_data, t_data, len_s, len_t):
    if length == 0:
        return 0, 0

    s_pref1, s_pref2, s_power1, s_power2 = s_data

```



```

t_pref1, t_pref2, t_power1, t_power2 = t_data

hashes = {}

for i in range(len_s - length + 1):
    h1 = get_hash(s_pref1, s_power1, i, length, MOD1)
    h2 = get_hash(s_pref2, s_power2, i, length, MOD2)
    hashes[(h1, h2)] = i

for j in range(len_t - length + 1):
    h1 = get_hash(t_pref1, t_power1, j, length, MOD1)
    h2 = get_hash(t_pref2, t_power2, j, length, MOD2)

    if (h1, h2) in hashes:
        return hashes[(h1, h2)], j

return -1, -1

def solve_pair(s, t):
    len_s = len(s)
    len_t = len(t)

    s_data = build_hashes(s)
    t_data = build_hashes(t)

    left = 0

```

```
right = min(len_s, len_t)

answer_i = 0
answer_j = 0
answer_len = 0

while left <= right:
    mid = (left + right) // 2

    i, j = check(mid, s_data, t_data, len_s, len_t)

    if i != -1:
        answer_i = i
        answer_j = j
        answer_len = mid
        left = mid + 1
    else:
        right = mid - 1

return answer_i, answer_j, answer_len
```

```
def solve():
    with open("input.txt", "r", encoding="utf-8") as f:
        lines = f.read().splitlines()

    result = []
```

```
for line in lines:

    if not line.strip():

        continue

    s, t = line.split()

    i, j, length = solve_pair(s, t)

    result.append(f"{i} {j} {length}")

with open("output.txt", "w", encoding="utf-8") as f:

    f.write("\n".join(result) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо найти наибольшую общую подстроку двух строк. Прямое динамическое программирование работает за $O(|s| * |t|)$, что может быть слишком медленно при больших строках. Поэтому используется хеширование и бинарный поиск по длине ответа.

Сначала для каждой строки строятся префиксные хеши по двум модулям. Это позволяет получать хеш любой подстроки за $O(1)$.

Далее выполняется бинарный поиск по длине общей подстроки. Для некоторой длины `length` программа проверяет, существует ли общая подстрока такой длины в обеих строках.

Для проверки все хеши подстрок длины `length` из строки `s` заносятся в словарь. Затем программа перебирает все подстроки такой же длины в строке `t` и проверяет, есть ли их хеш в словаре. Если совпадение найдено, значит общая подстрока такой длины существует.

Если общая подстрока длины `mid` найдена, программа пытается увеличить длину и сдвигает левую границу бинарного поиска. Если не найдена — уменьшает правую границу.

Используются два модуля:

`MOD1 = 1000000007`

`MOD2 = 1000000009`

Это уменьшает вероятность коллизий. В результате программа находит длину наибольшей общей подстроки и позиции ее начала в обеих строках.

Сложность решения составляет примерно $O((|s| + |t|) * \log(\min(|s|, |t|)))$, что подходит для данной задачи.

.

Таблица тестирования:

время	память	input	output
		cool toolbox	1 1 3

0.001433 sec	10.45 KB		
0.001130 sec	10.43 KB	aaa bb	0 0 0
0.001234 sec	10.45 KB	aabaa babbaab	2 3 3
0.001250 sec	10.45 KB	banana ananas	1 0 5

Пояснение к тестам

В первом тесте строки cool и toolbox. Их наибольшая общая подстрока - ool, она начинается с позиции 1 в обеих строках. Поэтому программа выводит 1 1 3.

Во втором тесте строки aaa и bb не имеют общих символов. Поэтому длина общей подстроки равна 0, и программа выводит 0 0 0.

В третьем тесте строки aabaa и babbaab. Программа нашла общую подстроку баа длины 3. В первой строке она начинается с позиции 2, а во второй строке - с позиции 3. Ответ корректный, так как требуется вывести любую наибольшую общую подстроку.

В четвертом тесте строки banana и ananas. Их наибольшая общая подстрока - анапа, она начинается с позиции 1 в строке banana и с позиции 0 в строке ananas. Поэтому программа выводит 1 0 5.

Вывод по задаче

В ходе решения задачи был реализован алгоритм поиска наибольшей общей подстроки с использованием бинарного поиска и полиномиального хеширования. Префиксные хеши позволяют быстро получать хеши подстрок, а словарь используется для проверки совпадений между двумя строками. Программа корректно находит длину максимальной общей

подстроки и позиции ее начала в обеих строках. Если подходящих подстрок максимальной длины несколько, решение может вывести любую из них, что допускается условием задачи. Использование двух модулей снижает вероятность коллизий и делает алгоритм достаточно надежным.

Задача №8. Шаблоны с несовпадениями

Текст задачи:

Даны число k , текст t и паттерн p . Нужно найти все позиции в тексте t , начиная с которых паттерн p входит в текст с не более чем k несовпадениями.

То есть для каждой позиции i нужно сравнить строку p со строкой:

$t[i : i + \text{len}(p)]$

Если количество отличающихся символов не больше k , то позиция i подходит.

Во входном файле может быть несколько строк. Каждая строка содержит:

$k \ t \ p$

Для каждой строки входных данных нужно вывести количество подходящих позиций и сами позиции. Индексация начинается с нуля.

Ограничения:

$$0 \leq k \leq 5$$

$$1 \leq |t| \leq 200000$$

$$1 \leq |p| \leq \min(|t|, 100000)$$

Код:

```
import time
import tracemalloc
```

```
MOD1 = 1_000_000_007
```

```
MOD2 = 1_000_000_009
```

```
X = 263
```

```
def build_hashes(s):
```

```
    n = len(s)
```

```
    power1 = [1] * (n + 1)
```

```
    power2 = [1] * (n + 1)
```

```
    pref1 = [0] * (n + 1)
```

```
    pref2 = [0] * (n + 1)
```

```
    for i in range(1, n + 1):
```

```
        power1[i] = (power1[i - 1] * X) % MOD1
```

```
        power2[i] = (power2[i - 1] * X) % MOD2
```

```
    for i in range(n):
```

```
        value = ord(s[i])
```

```
        pref1[i + 1] = (pref1[i] * X + value) % MOD1
```

```
        pref2[i + 1] = (pref2[i] * X + value) % MOD2
```

```
    return pref1, pref2, power1, power2
```

```

def get_hash(pref, power, left, length, mod):

    return (pref[left + length] - pref[left] * power[length]) % mod


def equal_substrings(t_data, p_data, t_left, p_left, length):

    t_pref1, t_pref2, t_power1, t_power2 = t_data
    p_pref1, p_pref2, p_power1, p_power2 = p_data

    h1_t = get_hash(t_pref1, t_power1, t_left, length, MOD1)
    h1_p = get_hash(p_pref1, p_power1, p_left, length, MOD1)

    if h1_t != h1_p:

        return False

    h2_t = get_hash(t_pref2, t_power2, t_left, length, MOD2)
    h2_p = get_hash(p_pref2, p_power2, p_left, length, MOD2)

    return h2_t == h2_p


def count_mismatches(t, p, t_data, p_data, start, k):

    m = len(p)

    pos = 0

    mismatches = 0

    while pos < m:

```



```
    left = 0

    right = m - pos

    same = 0

    while left <= right:

        mid = (left + right) // 2

        if equal_substrings(t_data, p_data, start + pos, pos,
mid):

            same = mid

            left = mid + 1

        else:

            right = mid - 1

    pos += same

    if pos == m:

        break

    mismatches += 1

    if mismatches > k:

        return mismatches

    pos += 1

return mismatches
```

```
def solve_case(k, t, p):

    t_data = build_hashes(t)

    p_data = build_hashes(p)

    result = []

    n = len(t)

    m = len(p)

    for start in range(n - m + 1):

        mismatches = count_mismatches(t, p, t_data, p_data, start, k)

        if mismatches <= k:

            result.append(start)

    return result


def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        lines = f.read().splitlines()

    output = []

    for line in lines:

        if not line.strip():
```

```
        continue

    parts = line.split()

    k = int(parts[0])

    t = parts[1]

    p = parts[2]

    result = solve_case(k, t, p)

    if result:

        output.append(str(len(result)) + " " + " ".join(map(str,
result)))

    else:

        output.append("0")

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write("\n".join(output) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()
```

```
with open("stats.txt", "w", encoding="utf-8") as f:
    f.write(f"time = {finish_time - start_time:.6f} sec\n")
    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче необходимо найти все позиции, где паттерн встречается в тексте с небольшим количеством несовпадений. Так как k не больше 5, можно для каждой позиции искать не все различия подряд, а только следующие несовпадающие символы.

Сначала для текста t и паттерна p строятся префиксные хеши по двум модулям. Это позволяет быстро сравнивать любые две подстроки одинаковой длины.

Для каждой позиции $start$ в тексте программа сравнивает паттерн с подстрокой текста, начинающейся в этой позиции. Чтобы найти следующее несовпадение, используется бинарный поиск. Он определяет максимальную длину общего совпадающего префикса между оставшейся частью паттерна и соответствующей частью текста.

Если вся оставшаяся часть совпала, значит больше несовпадений нет. Если совпала только часть, то следующий символ считается несовпадением, счетчик `mismatches` увеличивается, и проверка продолжается дальше.

Если количество несовпадений стало больше k , текущая позиция сразу отбрасывается. Если после проверки всей строки количество несовпадений не превышает k , позиция добавляется в ответ.

Такой подход работает эффективно, потому что для каждой позиции выполняется не полное сравнение строк, а не более $k + 1$ поисков следующего несовпадения. Каждый такой поиск выполняется бинарным поиском по длине с использованием хешей.

Сложность решения примерно равна:

$$O((|t| - |p| + 1) * (k + 1) * \log |p|)$$

Так как $k \leq 5$, решение укладывается в ограничения.

Таблица тестирования:

время	память	input	output
0.002450 sec	10.46 KB	0 ababab baaa	0
0.001543 sec	10.54 KB	1 ababab baaa	1 1
0.001787 sec	10.54 KB	2 хabcabc ccc	4 1 2 3 4
0.001068 sec	10.51 KB	3 aaa xxx	1 0

Пояснение к тестам

В первом тесте $k = 0$, поэтому нужно искать только точные совпадения. В тексте ababab паттерн baaa точно не встречается, поэтому ответ равен 0.

Во втором тесте разрешено одно несовпадение. Паттерн baaa подходит к подстроке текста, начинающейся с позиции 1, поэтому программа выводит 1 1.

В третьем тесте разрешено два несовпадения. В строке хabcabc паттерн ccc подходит к подстрокам, начинающимся с позиций 1, 2, 3 и 4, так как в каждой из них не больше двух отличающихся символов.

В четвертом тесте сравниваются строки aaa и xxx. Они отличаются во всех трех позициях, но так как $k = 3$, такое вхождение подходит. Поэтому ответ 1 0

Вывод по задаче

В ходе решения задачи был реализован алгоритм поиска вхождений паттерна в текст с ограниченным числом несовпадений. Для быстрого сравнения частей строк используются префиксные хеши по двум модулям. Следующее несовпадение находится с помощью бинарного поиска по длине совпадающего фрагмента. Благодаря тому, что число допустимых несовпадений мало, программа не сравнивает каждую позицию полностью и работает достаточно эффективно. Решение корректно обрабатывает точный поиск, поиск с одним или несколькими несовпадениями, а также случай, когда все символы паттерна отличаются от соответствующей подстроки текста.

Задача №9. Декомпозиция строки

Текст задачи:

Дана строка s , состоящая из букв латинского алфавита. Нужно построить самое короткое описание этой строки с помощью повторений подстрок.

Например, строку:

ABCABCDEDEDEF

можно записать короче как:

$ABC*2+DE*3+F$

Здесь запись $ABC*2$ означает, что строка ABC повторяется два раза подряд. Если множитель равен 1, то $*1$ можно не писать.

Нужно разбить исходную строку на несколько частей и каждую часть записать либо как обычную строку, либо как повторение строки. Между частями ставится знак +. Требуется минимизировать длину итогового описания. Если оптимальных ответов несколько, можно вывести любой.

Ограничение:

$$1 \leq |s| \leq 5000$$

Код:

```
import time

import tracemalloc

MOD1 = 1000000007
MOD2 = 1000000009
X = 263

def build_hashes(s):
    n = len(s)

    power1 = [1] * (n + 1)
    power2 = [1] * (n + 1)

    pref1 = [0] * (n + 1)
    pref2 = [0] * (n + 1)
```

```

for i in range(1, n + 1):

    power1[i] = (power1[i - 1] * X) % MOD1

    power2[i] = (power2[i - 1] * X) % MOD2


for i in range(n):

    value = ord(s[i])

    pref1[i + 1] = (pref1[i] * X + value) % MOD1

    pref2[i + 1] = (pref2[i] * X + value) % MOD2


return pref1, pref2, power1, power2


def get_hash(pref, power, left, length, mod):

    return (pref[left + length] - pref[left] * power[length]) % mod


def equal_substrings(data, a, b, length):

    pref1, pref2, power1, power2 = data

    h1_a = get_hash(pref1, power1, a, length, MOD1)

    h1_b = get_hash(pref1, power1, b, length, MOD1)

    if h1_a != h1_b:

        return False

    h2_a = get_hash(pref2, power2, a, length, MOD2)

```



```
h2_b = get_hash(pref2, power2, b, length, MOD2)

return h2_a == h2_b


def is_period(data, left, length, period):

    return equal_substrings(data, left, left + period, length -
period)


def make_divisors(n):

    divs = [[] for _ in range(n + 1)]

    for d in range(1, n + 1):

        for length in range(d * 2, n + 1, d):

            divs[length].append(d)

    return divs


def get_best_term(s, data, divs, left, right):

    length = right - left

    best_period = length

    best_count = 1

    best_cost = length
```

```

for period in divs[length]:

    if is_period(data, left, length, period):

        count = length // period

        cost = period + 1 + len(str(count))

        if cost < best_cost:

            best_cost = cost

            best_period = period

            best_count = count

return best_cost, best_period, best_count

def build_answer(s, parent):

    parts = []

    pos = len(s)

    while pos > 0:

        prev, period, count = parent[pos]

        if count == 1:

            parts.append(s[prev:pos])

        else:

            parts.append(s[prev:prev + period] + "*" + str(count))

        pos = prev

```

```

parts.reverse()

return "+".join(parts)

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        s = f.read().strip()

    n = len(s)

    data = build_hashes(s)

    divs = make_divisors(n)

    INF = 10 ** 18

    dp = [INF] * (n + 1)

    parent = [None] * (n + 1)

    dp[0] = 0

    for right in range(1, n + 1):

        for left in range(right):

            term_cost, period, count = get_best_term(s, data, divs,
left, right)

            plus = 1 if left > 0 else 0

            current = dp[left] + plus + term_cost

```

```

        if current < dp[right]:

            dp[right] = current

            parent[right] = (left, period, count)

    answer = build_answer(s, parent)

    with open("output.txt", "w", encoding="utf-8") as f:

        f.write(answer + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")

```

Текстовое объяснение решения: В этой задаче нужно найти наиболее короткое описание строки. Для этого используется динамическое программирование по префиксам строки.

Пусть $dp[i]$ - минимальная длина описания для префикса строки $s[0:i]$. Тогда для каждой позиции $right$ перебирается предыдущая граница $left$. Подстрока $s[left:right]$ рассматривается как последняя часть описания.

Для каждой такой подстроки программа определяет, можно ли записать ее короче через повторение некоторого блока. Например, строку `abababab` можно записать как `ab*4`. Для проверки повторяемости используются полиномиальные хеши. Если подстрока имеет период $period$, то достаточно проверить, что:

$$s[left : right - period] = s[left + period : right]$$

Если это равенство выполняется и длина подстроки делится на $period$, значит подстроку можно представить как повторение блока длины $period$.

Стоимость обычной записи подстроки равна ее длине. Стоимость записи с повторением равна:

длина блока + 1 + количество цифр в множителе

Единица добавляется за символ `*`.

После этого программа обновляет значение $dp[right]$. Если перед текущей частью уже была другая часть, то дополнительно учитывается знак `+`.

Для восстановления ответа хранится массив `parent`. В нем запоминается, откуда пришли в текущую позицию, какой период был выбран и сколько раз повторяется блок. После заполнения динамики ответ восстанавливается с конца строки к началу.

Сложность решения зависит от перебора подстрок и их возможных периодов. Для ускорения сравнения подстрок используются заранее посчитанные префиксные хеши, поэтому каждая проверка равенства выполняется за $O(1)$.

.

Таблица тестирования:

время	память	input	output
-------	--------	-------	--------

0.002434 sec	14.00 KB	ABCABCDEDEDEF	ABC*2+DE*3+F
0.001410 sec	11.34 KB	Hello	Hello
0.002144 sec	12.68 KB	ababababx	ab*4+x
0.002949 sec	13.67 KB	aaaaabaaaaab	aaaaab*2

Пояснение к тестам

В первом тесте строка ABCABCDEDEDEF. В ней подстрока ABC повторяется два раза подряд, а подстрока DE повторяется три раза подряд. Поэтому строку можно записать как ABC*2+DE*3+F.

Во втором тесте строка Hello не содержит подряд идущих повторяющихся блоков, которые позволили бы уменьшить запись. Поэтому оптимальное представление совпадает с исходной строкой.

В третьем тесте строка ababababx. Первая часть abababab состоит из четырех повторений строки ab, поэтому ее можно записать как ab*4. Последний символ x добавляется отдельно.

В четвертом тесте строка aaaaabaaaaab состоит из двух повторений блока aaaaab, поэтому оптимальная запись имеет вид aaaaab*2.

Вывод по задаче

В ходе решения задачи был реализован алгоритм построения короткого представления строки с помощью динамического программирования и хеширования. Динамика позволяет выбрать оптимальное разбиение строки на части, а хеши используются для быстрой проверки того, является ли подстрока повторением меньшего блока. Программа корректно обрабатывает строки без повторений, строки с одним повторяющимся

блоком и строки, которые выгодно разбивать на несколько частей. Решение восстанавливает само оптимальное представление, а не только его длину.

Дополнительная задача №1. Подстроки из одинаковых букв

Текст задачи:

Дана строка s , состоящая из строчных латинских букв. Нужно посчитать количество подстрок, которые состоят только из одинаковых символов.

Подстрокой называется непрерывный фрагмент строки.

Например, в строке:

aaabb

подстроки из одинаковых букв:

a, a, a, aa, aa, aaa, b, b, bb

Всего таких подстрок 9.

Нужно вывести одно число - количество подстрок, состоящих из одинаковых букв.

Код:

```
import time

import tracemalloc

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        s = f.read().strip()
```

```
ans = 0

current_len = 0

last_char = ""

for ch in s:

    if ch == last_char:

        current_len += 1

    else:

        current_len = 1

        last_char = ch

    ans += current_len

with open("output.txt", "w", encoding="utf-8") as f:

    f.write(str(ans) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()
```



```
with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче нужно посчитать количество подстрок, состоящих только из одинаковых символов.

Подстрока из одинаковых букв полностью находится внутри блока одинаковых подряд идущих символов. Например, если в строке есть блок аааа длины 4, то внутри него можно выбрать:

4 подстроки длины 1,

3 подстроки длины 2,

2 подстроки длины 3,

1 подстроку длины 4.

Всего получается:

$$4 + 3 + 2 + 1 = 10$$

В программе строка просматривается слева направо. Переменная `current_len` хранит длину текущего блока одинаковых символов, который заканчивается в текущей позиции.

Если текущий символ совпадает с предыдущим, длина блока увеличивается на 1. Если символ изменился, начинается новый блок длины 1.

На каждом шаге к ответу добавляется `current_len`, потому что ровно столько подстрок из одинаковых букв заканчивается в текущей позиции.

Например, для строки ааа:

на первой букве заканчивается 1 подходящая подстрока: а;

на второй букве заканчиваются 2 подстроки: а, аа;

на третьей букве заканчиваются 3 подстроки: a, aa, aaa.

Итого получается $1 + 2 + 3 = 6$.

Алгоритм работает за $O(n)$, где n — длина строки. Дополнительная память используется только для нескольких переменных, поэтому сложность по памяти равна $O(1)$.

Таблица тестирования:

время	память	input	output
0.001380 sec	10.24 KB	aaabb	9
0.001384 sec	10.24 KB	abcde	5
0.000979 sec	10.24 KB	aaaaa	15
0.000941 sec	10.24 KB	aabbbaac	13

Пояснение к тестам

В первом тесте строка aaabb. Она разбивается на два блока одинаковых символов: aaa и bb. Блок aaa дает 6 подходящих подстрок, а блок bb дает 3 подходящие подстроки. Всего получается $6 + 3 = 9$.

Во втором тесте строка abcde не содержит соседних одинаковых символов. Поэтому каждая подходящая подстрока имеет длину 1, и ответ равен количеству символов в строке, то есть 5.

В третьем тесте строка aaaaa полностью состоит из одинаковых символов. Количество подстрок из одинаковых букв равно $5 + 4 + 3 + 2 + 1 = 15$.

В четвертом тесте строка aabbbaas разбивается на блоки aa, bb, aaa и c. Эти блоки дают соответственно 3, 3, 6 и 1 подходящую подстроку. Всего получается $3 + 3 + 6 + 1 = 13$

Вывод по задаче

В ходе решения задачи был реализован линейный алгоритм подсчета подстрок, состоящих из одинаковых символов. Программа последовательно проходит по строке и хранит длину текущего блока одинаковых букв. На каждой позиции к ответу добавляется количество подходящих подстрок, которые заканчиваются в этой позиции. Решение корректно обрабатывает строки без повторов, строки из одного повторяющегося символа и строки с несколькими блоками одинаковых букв. Алгоритм работает за линейное время и использует постоянную дополнительную память.

Дополнительная задача №2. Количество различных подстрок

Текст задачи:

Дана строка s , состоящая из строчных латинских букв. Нужно посчитать количество различных подстрок этой строки.

Подстрокой называется непрерывный фрагмент строки. Если одна и та же подстрока встречается несколько раз, она считается только один раз.

Например, для строки:

aaa

различные подстроки:

a

aa

aaa

Всего их 3.

Нужно вывести одно число - количество различных подстрок строки.

Код:

```
import time

import tracemalloc

MOD1 = 1000000007
MOD2 = 1000000009
X = 263

def get_hash(pref, power, left, length, mod):

    return (pref[left + length] - pref[left] * power[length]) % mod

def solve():

    with open("input.txt", "r", encoding="utf-8") as f:

        s = f.read().strip()

    n = len(s)

    power1 = [1] * (n + 1)
    power2 = [1] * (n + 1)

    for i in range(1, n + 1):
```

```

        power1[i] = (power1[i - 1] * X) % MOD1

        power2[i] = (power2[i - 1] * X) % MOD2

    pref1 = [0] * (n + 1)
    pref2 = [0] * (n + 1)

    for i in range(n):
        value = ord(s[i])

        pref1[i + 1] = (pref1[i] * X + value) % MOD1
        pref2[i + 1] = (pref2[i] * X + value) % MOD2

    substrings = set()

    for length in range(1, n + 1):
        for left in range(n - length + 1):
            h1 = get_hash(pref1, power1, left, length, MOD1)
            h2 = get_hash(pref2, power2, left, length, MOD2)
            substrings.add((h1, h2, length))

    with open("output.txt", "w", encoding="utf-8") as f:
        f.write(str(len(substrings)) + "\n")

tracemalloc.start()

start_time = time.perf_counter()

solve()

```

```
finish_time = time.perf_counter()

cur, peak = tracemalloc.get_traced_memory()

tracemalloc.stop()

with open("stats.txt", "w", encoding="utf-8") as f:

    f.write(f"time = {finish_time - start_time:.6f} sec\n")

    f.write(f"memory = {peak / 1024:.2f} KB\n")
```

Текстовое объяснение решения: В этой задаче нужно посчитать количество различных подстрок заданной строки. Одинаковые подстроки, которые встречаются в разных местах строки, должны учитываться только один раз.

Для решения используется хеширование строк. Сначала для исходной строки строятся префиксные хеши и массивы степеней основания X . Это позволяет получать хеш любой подстроки за $O(1)$.

Далее программа перебирает все возможные длины подстрок от 1 до n . Для каждой длины перебираются все позиции начала подстроки. Для каждой подстроки вычисляются два хеша по разным модулям:

$MOD1 = 1000000007$

$MOD2 = 1000000009$

Пара хешей вместе с длиной подстроки добавляется в множество `set`. Множество автоматически хранит только уникальные элементы, поэтому одинаковые подстроки не будут посчитаны несколько раз.

Длина подстроки добавляется в множество вместе с хешами, чтобы подстроки разной длины точно считались разными объектами.

После перебора всех подстрок ответом является размер множества.

Сложность алгоритма составляет $O(n^2)$, так как всего у строки порядка n^2 подстрок. Получение хеша каждой подстроки выполняется за $O(1)$. Использование двух модулей снижает вероятность коллизий.

Таблица тестирования:

время	память	input	output
0.001190 sec	11.01 KB	aaa	3
0.001403 sec	12.29 KB	abcd	10
0.001368 sec	12.45 KB	ababa	9
0.001845 sec	13.30 KB	banana	15

Пояснение к тестам

В первом тесте строка aaa. Ее различные подстроки: а, аа, ааа, поэтому ответ равен 3.

Во втором тесте строка abcd. Все символы разные, и повторяющихся подстрок нет. Общее количество подстрок строки длины 4 равно $4 * 5 / 2 = 10$.

В третьем тесте строка ababa. В ней есть повторяющиеся подстроки, например а, b, ab, ba, aba. После удаления повторов остается 9 различных подстрок.

В четвертом тесте строка banana. Несмотря на то что в ней есть повторяющиеся буквы и фрагменты, всего получается 15 различных подстрок.

Вывод по задаче

В ходе решения задачи был реализован алгоритм подсчета различных подстрок строки с использованием полиномиального хеширования. Программа перебирает все возможные подстроки, вычисляет для каждой пару хешей и сохраняет их во множество. Благодаря множеству одинаковые подстроки учитываются только один раз. Решение корректно работает для строк из одинаковых символов, строк без повторений и строк с повторяющимися фрагментами. Использование двух модулей уменьшает вероятность ошибки из-за совпадения хешей.

Общий вывод по работе

В ходе выполнения лабораторной работы были изучены и реализованы алгоритмы обработки строк и поиска подстрок. Основное внимание было уделено наивному поиску, алгоритму Рабина-Карпа, сравнению подстрок с помощью хеширования, построению префикс-функции и Z-функции, а также более сложным задачам на поиск наибольшей общей подстроки и работу с шаблонами с несовпадениями.

В соответствии с вариантом 21 были выполнены обязательные задачи №3, №4 и №7. Для добора баллов дополнительно были решены задачи №1, №2, №5, №6, №8 и №9, а также две дополнительные задачи: “Подстроки из одинаковых букв” и “Количество различных подстрок”.

В процессе работы были использованы разные подходы к обработке строк. В простых задачах применялись прямой перебор и подсчет с помощью массивов частот. В задачах с большими ограничениями использовались полиномиальные хеши, что позволило быстро сравнивать подстроки и находить совпадения. Для уменьшения вероятности коллизий в нескольких задачах применялось двойное хеширование по двум большим модулям. Также были реализованы линейные алгоритмы построения префикс-функции и Z-функции.

Все программы считывают данные из файла input.txt, записывают результат в output.txt, а также дополнительно сохраняют время выполнения и используемую память в файл stats.txt. Для каждой задачи были проведены тесты, проверяющие обычные случаи, отсутствие совпадений, повторяющиеся символы, пересекающиеся вхождения и другие важные ситуации.

В результате выполнения лабораторной работы были закреплены навыки эффективной работы со строками, поиска подстрок и применения хеширования. Реализованные алгоритмы укладываются в заданные ограничения по времени и памяти и корректно работают на рассмотренных тестовых данных.

.